

How attackers drain non-EVM blockchain wallets

Non-EVM blockchains face fundamentally different security threats than Ethereum, with **Solana** experiencing sophisticated drainer operations stealing millions**, while chains like Cardano and NEAR have largely avoided wallet draining through architectural design choices.

[Phemex](<https://phemex.com/academy/cryptocurrency-glossary/bridge-attacks>) In 2024, cryptocurrency losses totaled \$2.0-2.4 billion across all chains, [medium +5](<https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24>) with **85.3%** of wallet drainer attacks targeting Ethereum** (\$494M total from drainers),

[medium](<https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24>) yet non-EVM chains faced \$542.7M in losses from centralized exchange breaches, [The Block](<https://www.theblock.co/post/331626/crypto-hacks-exploits-2024>)

[Decrypt](<https://decrypt.co/297206/largest-crypto-exploits-hacks-2024>) emerging drainer campaigns on Solana (\$900K+ confirmed from one operation), [Google Cloud](<https://cloud.google.com/blog/topics/threat-intelligence/solana-cryptocurrency-stolen-clinksink-drainer-campaigns>) and a critical Tron vulnerability affecting 14,500+ wallets with \$31.5M at risk. [Coingecko +4](<https://coingecko.com/news/tron-addresses-risk-silent-hijacking>) The key distinction: while EVM chains suffer from unlimited token approval exploits, non-EVM chains employ diverse security models—from Solana's delegation system to NEAR's function-call access keys to Cardano's complete elimination of approval mechanisms—each creating unique attack surfaces that require chain-specific security approaches.

Solana leads non-EVM draining attacks with sophisticated exploits

Solana represents the most active non-EVM target for wallet draining operations, driven by its unique architecture that creates attack vectors absent from other blockchains. [Google Cloud](<https://cloud.google.com/blog/topics/threat-intelligence/solana-cryptocurrency-stolen-clinksink-drainer-campaigns>) The platform processed over **\$4.17 million** in confirmed drainer thefts from 3,947 victims** between 2023-2024, with operations like CLINKSINK generating at least \$900,000 through 35+ affiliate networks targeting Phantom wallet users. [Google Cloud](<https://cloud.google.com/blog/topics/threat-intelligence/solana-cryptocurrency-stolen-clinksink-drainer-campaigns>) [google](<https://cloud.google.com/blog/topics/threat-intelligence/solana-cryptocurrency-stolen-clinksink-drainer-campaigns>)

****SPL token delegation creates persistent attack surface**.** Unlike Ethereum's contract-based ERC-20 tokens, Solana uses a single shared Token Program managing all SPL tokens through separate account structures. [Thogiti](<https://thogiti.github.io/2025/02/26/SPL-ERC20-Tokens.html>)

[Solana](https://solana.com/docs/tokens/basics/revoke-delegate) Each token account supports only **one** delegate at a time with a specified delegated amount, but critically, this delegation persists indefinitely until explicitly revoked. When users approve malicious dApps, attackers gain the ability to transfer tokens at any future point without additional authorization. The single-delegate limitation means new approvals overwrite previous ones without creating an audit trail, [Stack Exchange](https://solana.stackexchange.com/questions/7715/active-delegates-on-a-spl-token-account) and delegates remain active even if account balances drop below delegated amounts. [Stack Exchange](https://solana.stackexchange.com/questions/755/are-spl-token-delegates-exclusive-transfer-authorities) [Solana](https://spl.solana.com/token)

Program Derived Addresses enable state manipulation attacks. PDAs are addresses derived deterministically from program IDs and seed values, existing off the Ed25519 elliptic curve so no private key controls them. [Stack Exchange](https://solana.stackexchange.com/questions/26/what-is-a-program-derived-address-pda-exactly) [Ackee](https://ackee.xyz/solana/book/latest/chapter3/program-derived-address/) Programs can sign transactions for PDAs using `invoke_signed()`, enabling legitimate cross-program invocations. [Chainstack](https://chainstack.com/solana-how-to-use-program-derived-addresses-for-state-management/) [Alchemy](https://www.alchemy.com/overviews/program-derived-address) However, attackers exploit this by deploying malicious programs with PDA-controlled state accounts. During transaction simulation, the PDA state appears benign. After users sign, attackers modify PDA state via separate transactions before execution. Solana's 400ms block times [Solana](https://solana.com/docs/core/transactions) enable rapid state changes between simulation and execution, creating **time-of-check-time-of-use (TOCTOU) vulnerabilities** that traditional simulation cannot prevent.

Blind signing exposes users to hidden instructions. Solana transactions are base64-encoded blobs requiring signatures over opaque data. [Solana](https://solana.com/docs/core/transactions) [Solana Wiki](https://solana.wiki/docs/solidity-guide/transactions/) Wallets show simulated behavior, not guaranteed execution outcomes. [Stack Exchange](https://solana.stackexchange.com/questions/7573/how-can-a-wallet-drainer-work-if-i-dont-sign-anything) The transaction structure includes a message with header, account keys array, recent blockhash, and compiled instructions—all within a 1,232-byte limit. [Solana](https://solana.com/docs/core/transactions) Users signing these transactions grant permission for ALL programs in the instruction set to modify writable accounts, with no per-instruction authorization. Hardware wallets like Ledger require "blind signing" enabled for Solana dApps, [Phantom](https://help.phantom.com/hc/en-us/articles/16517984492947-How-to-enable-blind-signing-in-Ledger-Nano) [Agave](https://docs.anza.xyz/cli/examples/sign-offchain-message) fundamentally preventing clear transaction review. Attackers hide delegation approvals or transfer instructions within complex multi-program transactions that appear legitimate during simulation.

****Real-world Solana incidents demonstrate vulnerability severity**.** The ****Slope Wallet hack in August 2022**** compromised 9,231 wallets for \$4.1-6.7 million when the mobile wallet transmitted private keys in plaintext to Sentry monitoring servers stored on Alibaba Cloud. [Stack Exchange +4](https://solana.stackexchange.com/questions/7573/how-can-a-wallet-drainer-work-if-i-dont-sign-anything) The ****CLINKSINK drainer operation**** running December 2023 through early 2024 operated as drainer-as-a-service with an 80/20 affiliate split, deploying JavaScript code targeting Phantom wallet specifically. Operator wallet B8YldERnVNoUUXeXA4NaCHiB9htcukMSkFhrFsTMHA7h received 1,491+ SOL (\$180,000+), implying ~\$900,000 total stolen at the 20% operator rate. [Google Cloud](https://cloud.google.com/blog/topics/threat-intelligence/solana-cryptocurrency-stolen-clinksink-drainer-campaigns) ****March 2024 saw 302 victims lose 2,808 SOL (~\$523,000)****, with the largest single loss exceeding 500 SOL (\$93,000), traced to private key exports used in Telegram trading bots.

****Drainer-as-a-service ecosystem thrives on Solana**.** The largest Solana drainer Telegram community has ****6,000+ members****. [CCN](https://www.ccn.com/news/more-than-6000-people-join-solana-drainer-community/) [Cointelegraph](https://cointelegraph.com/news/solana-wallet-draiders-target-users-amid-rise-in-sol-price) Pricing ranges from \$250/month subscriptions to \$100-500 one-time purchases, [Cointelegraph](https://cointelegraph.com/news/solana-wallet-draiders-target-users-amid-rise-in-sol-price) [CCN](https://www.ccn.com/news/more-than-6000-people-join-solana-drainer-community/) with premium operations like Angel Drainer charging \$5,000-10,000 upfront plus 20% commission. [Cointelegraph](https://cointelegraph.com/explained/crypto-draiders-as-a-service-what-you-need-to-know) Drainer kits include wallet balance enumeration, asset valuation to prioritize high-value tokens, automatic transaction creation, simulation bypass capabilities, and bit-flip attack features (Aqua and Vanish draiders). The Russian-language development community maintains documentation, [CryptoRank.io](https://cryptorank.io/news/feed/a48a0-growing-concerns-over-solana) [Cointelegraph](https://cointelegraph.com/news/solana-wallet-draiders-target-users-amid-rise-in-sol-price) demonstration videos, and technical support channels. Geographic concentration enables rapid evolution to evade detection tools like Wallet Guard and Blockaid.

Architecture determines vulnerability across non-EVM chains

Different blockchain architectures create fundamentally distinct attack surfaces, with some designs eliminating entire classes of vulnerabilities present on EVM chains while introducing novel risks.

****Cosmos ecosystem resists approval-based draining through design**.** Cosmos SDK chains have ****no EVM-style token approval mechanism****—no equivalent to approve() and transferFrom() functions. Each IBC token transfer requires explicit user signature, eliminating persistent spending permissions that enable wallet

drainers. [Cosmos Network](https://tutorials.cosmos.network/academy/2-cosmos-concepts/14-interchain-security.html) The architecture focuses security concerns on protocol and IBC vulnerabilities rather than user wallet exploitation. The ****Osmosis bug in June 2022**** that allowed 50% bonus withdrawals from liquidity pools threatened \$212.8M TVL but validators halted the chain limiting losses to \$5 million. [The Block +2](https://www.theblock.co/post/150752/osmosis-dex-on-cosmos-exploited-for-5-million-as-validators-halt-the-network) Multiple critical IBC vulnerabilities discovered by security researchers (reentrancy attacks potentially risking \$126M+, event hallucination bugs affecting bridges, Dragonberry timeout forgery) were all patched before exploitation. [Coingecko +2](https://coingecko.com/news/cosmos-patches-bug-ibc-protocol-saves-126m) CosmWasm smart contracts use a submessages pattern, and many chains require governance approval for contract deployment, implementing defense-in-depth that prevented real-world wallet draining despite theoretical attack paths. [Cosmwasm](https://cosmwasm.com/) [Cosmos Network](https://tutorials.cosmos.network/academy/2-cosmos-concepts/14-interchain-security.html)

****Cardano's UTXO model eliminates approval vulnerabilities entirely****. Extended UTXO architecture treats transactions as consuming specific unspent outputs rather than modifying account balances. Smart contracts are validator scripts that check whether spending conditions are met—they cannot autonomously transfer user assets. [SDLC Corp +2](https://sdlccorp.com/post/common-issues-with-trc-20-tokens/) ****Zero documented wallet draining attacks have occurred on Cardano**** at the protocol level. [Emurgo](https://www.emurgo.io/press-news/exploring-cardano-blockchain-security/) The system requires explicit transaction signing for each spend, with all outputs visible before user approval. [Cardano Developer Portal](https://developers.cardano.org/docs/smart-contracts/) [cexplorer](https://cexplorer.io/article/why-is-simple-to-drain-the-ethereum-wallet-unlike-the-cardano-wallet) Native tokens inherit ledger-level security rather than existing as separate smart contracts, eliminating the entire attack class of malicious token contracts. [IOHK](https://iohk.io/blog/posts/2025/05/09/native-tokens-in-the-extended-utxo-model/) [cexplorer](https://cexplorer.io/article/why-is-simple-to-drain-the-ethereum-wallet-unlike-the-cardano-wallet) Individual loss reports trace exclusively to compromised seed phrases from phishing websites, not protocol-level vulnerabilities. The October 2021 double satisfaction vulnerability affecting early dApps was a known development issue that IOG publicly documented with available patches, with zero confirmed exploits in the wild.

****Move-based chains prevent resource duplication through type system****. Aptos and Sui implement resource-oriented programming where digital assets have copy, drop, key, and store abilities explicitly controlled. Resources without copy ability cannot be duplicated, preventing double-spending at the language level. Resources without drop must be explicitly handled, preventing accidental asset loss. Each resource has exactly one owner encoded in the type system, eliminating complex access control logic. This ****prevents reentrancy attacks**** (resources are moved, not referenced), integer overflow (strict arithmetic checks), and unauthorized access (ownership embedded in types). [Pontem](https://docs.pontem.network/02.

-move-language/lang) However, the **Cetus Protocol exploit on Sui in May 2024** demonstrated that protocol-level vulnerabilities remain. Attackers exploited arithmetic overflow in CLMM liquidity calculations combined with oracle manipulation to drain **\$220-260 million** from 46 liquidity pools, with \$162M frozen by validator governance vote and \$60M bridged to Ethereum. [CCN +2](https://ccn.com/education/crypto-hacks-exploits-full-list-scams-vulnerabilities)

Algorand's rekeying creates unique vulnerability. The platform enables accounts to maintain static public addresses while rotating authoritative private spending keys through the rekey_to transaction field. [Coininspect Security +2](https://www.coininspect.com/blog/algorand-rekeying-attacks/) Legitimate uses include key rotation without fund movement and sophisticated custody solutions. However, smart signatures that fail to validate rekey_to allow complete account takeover. Coininspect security research revealed that vulnerable contracts permit attackers to submit approved transactions with rekey_to set to attacker addresses, permanently transferring account control. [Coininspect Security](https://www.coininspect.com/blog/algorand-rekeying-attacks/) [coininspect](https://www.coininspect.com/blog/algorand-rekeying-attacks/) The **MyAlgo wallet hack in February 2023** compromised ~25 high-value accounts for **\$9.2-9.6 million** in ALGO and USDC through suspected seed phrase exfiltration, though the precise attack vector was never definitively determined. [Decrypt +2](https://decrypt.co/122360/algorand-wallet-myalgo-urgest-users-withdraw-funds-9-2m-exploit) Algorand's layer-1 ASA token standard provides inherent security advantages—assets are protocol-level rather than contract-based, requiring explicit opt-in before receipt [Algorand](https://dev.algorand.co/getting-started/ethereum-to-algorand/) [Algorand Technologies](https://algorandtechnologies.com/news/how-to-tokenize-assets-on-blockchain/) and preventing entire classes of ERC-20 vulnerabilities.

Tron inherits EVM vulnerabilities with unique permission exploit. TRC-20 tokens function identically to ERC-20, using approve() and transferFrom() mechanisms that create the same approval-based attack surface. [B2BINPAY](https://b2binpay.com/en/news/what-is-trc20-understanding-trons-token-standard-and-its-role-in-defi) [ECOS](https://ecos.am/en/blog/trc20-token-standard-understanding-trc20-in-the-tron-network/) Late 2024 revealed a critical **UpdateAccountPermission vulnerability** affecting 14,500+ wallets with \$31.5 million at risk. Attackers who obtain private keys (through phishing or malware) use the UpdateAccountPermission function to add their own keys to multi-signature configurations, setting thresholds that lock out legitimate owners. [Bitget +3](https://www.bitget.com/news/detail/12560604516689) The 100 TRX fee requirement provides no real barrier. Victims retain wallet access but cannot make outbound transactions, continuing to deposit funds unaware of compromise. [CoinTelegraph +3](https://cointelegraph.com/news/tron-addresses-risk-silent-hijacking) This silent hijacking method proved more insidious than immediate draining, with **2,130 wallets confirmed compromised in Q4 2024** [CoinTelegraph +2](https://cointelegraph.com/news/tron-addresses-risk-silent-hijacking) alone. [Bitget](https://www.bitget.com/news/detail/12560604516689) [Bitget](https://www.bitget.com/news/detail/12560604515174)

****TON's Telegram integration creates emerging threat landscape**.** The platform's Jetton token standard uses a two-contract system—a master contract managing metadata and minting, plus individual Jetton wallet contracts for each user. Deterministic wallet address calculation prevents some spoofing attacks, [The Open Network +2](https://docs.ton.org/v3/guidelines/dapps/asset-processing/jettons) but the ecosystem lacks mature security tooling. As Ethereum security tightened with Blockaid integration into MetaMask and Coinbase Wallet, drainers migrated to TON. [Invezz](https://invezz.com/news/2024/09/05/angelx-wallet-drafter-targets-vulnerable-ton-and-tron-blockchains/) [Airdrop Alert](https://airdropalert.com/blogs/ton-wallet-drafter-targets-vulnerable-ton-and-tron-blockchains/) ****One drafter earned 22,000 TON (\$152,000) in 2024**** exploiting TON's comment feature to trick victims into signing fraudulent transactions. Drafter scripts sold on Telegram for \$300-1,000 specifically target Tonkeeper wallet. [Airdrop Alert](https://airdropalert.com/blogs/ton-wallet-drafter-targets-vulnerable-ton-and-tron-blockchains/) The influx of millions of crypto novices from Telegram games like Notcoin and Hamster Kombat created a user base unfamiliar with seed phrase security, with TON daily active addresses surpassing Ethereum in 2024. [airdropalert](https://airdropalert.com/blogs/ton-wallet-drafter-targets-vulnerable-ton-and-tron-blockchains/) [Airdrop Alert](https://airdropalert.com/blogs/ton-wallet-drafter-targets-vulnerable-ton-and-tron-blockchains/)

****NEAR Protocol implements superior permission model**.** The platform's access key system separates full-access keys (complete account control) from function-call access keys (restricted to specific dApp interactions with gas allowances). When connecting to dApps, NEAR creates limited keys that only interact with that specific contract, have depleting gas allowances, cannot transfer NEAR tokens, and can be revoked anytime. [near +4](https://docs.near.org/concepts/protocol/access-keys) This design is ****superior to EVM approvals**** because compromise of one dApp doesn't affect others, and gas limits prevent indefinite draining. The ****June 2022 email/SMS recovery vulnerability**** exposed seed phrases to third-party services during password reset flows but was patched same-day with no confirmed fund losses, though NEAR delayed public disclosure until August when security firm Hacksyck revealed it following similarity to the Solana Slope hack. [Decrypt](https://decrypt.co/106819/near-protocol-wallet-breach-exposed-private-keys)

Statistics reveal Bitcoin and Ethereum dominate 2024 losses

Cross-chain data demonstrates that while EVM chains suffered the majority of wallet draining attacks, non-EVM chains experienced significant losses from centralized exchange breaches and emerging drafter operations.

****Total 2024 blockchain security losses reached \$2.0-2.4 billion**.** SlowMist reported \$2.013B across 410 incidents (down 19% from 2023), [medium](https://slowmist.medium.com/slowmist-2024-blockchain-)

security-and-anti-money-laundering-annual-report-d7fc94ccfb24) while CertiK reported \$2.363B across 760 incidents (up 32%). [CertiK +2](https://www.certik.com/resources/blog/hack3d-the-web3-security-report-2024) Chainalysis calculated \$2.2B in stolen funds. [Medium](https://slowmist.medium.com/analysis-of-the-2024-blockchain-security-and-anti-money-laundering-annual-report-security-248140ffb6bb) [Medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) The discrepancy stems from different methodologies—some sources include only hacks while others encompass scams and fraud, and token valuations vary by calculation time. ****Wallet drainer-specific losses totaled \$494 million**** affecting 332,000+ addresses (67% increase from 2023's \$295M), with 30 individual thefts exceeding \$1M each and the largest single theft reaching \$55.4 million. [medium +4](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24)

****Ethereum absorbed 85.3% of drainer attacks****. ScamSniffer data shows ****\$420M+ in drainer losses concentrated on Ethereum****, representing \$152M specifically from wallet draining operations with additional phishing signature losses of \$297.5M across 248 incidents. [The Block](https://blockchainnews.com/news/scammers-drained-usd-494-million-in-crypto-wallet-heists-in-2024/) [Bleeping Computer](https://www.bleepingcomputer.com/news/security/cryptocurrency-wallet-drainers-stole-494-million-in-2024/) The 403 Ethereum incidents averaged \$1.86M per exploit. [CertiK](https://www.certik.com/resources/blog/hack3d-the-web3-security-report-2024) [CertiK](https://www.certik.com/resources/blog/3gKPgxHukAE4bVW5J5Lvnq-hack3d-the-web3-security-report-2024) BSC experienced \$87.35M in losses across 149 incidents, representing a 61% decline from 2023's \$162M. [medium +2](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) Blast blockchain saw \$70M in losses with \$5.9M specifically from wallet drainers. Arbitrum lost \$3.6M to drainers. [Infosecurity Magazine +3](https://www.infosecurity-magazine.com/news/scammers-drain-500m-crypto-wallets/) The zkSync ecosystem suffered the highest rug pull losses at \$36.95M. [medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) [Crowdfund Insider](https://www.crowdfundinsider.com/2025/01/234920-defi-remains-most-frequently-targeted-crypto-and-web3-sector-for-attacks-slowmist-blockchain-security-report/)

****Bitcoin's largest incident came from centralized exchange compromise****. The ****DMM Bitcoin hack on May 31, 2024**** resulted in ****\$305 million**** (4,502.9 BTC) stolen [Halborn +3](https://www.halborn.com/blog/post/explained-the-dmm-bitcoin-hack-may-2024) through social engineering by the Lazarus Group. [The Block](https://www.theblock.co/post/331626/crypto-hacks-exploits-2024) [Decrypt](https://decrypt.co/297206/largest-crypto-exploits-hacks-2024) North Korean operatives using the TraderTraitor persona sent malicious Python scripts disguised as job coding tests via LinkedIn to a Ginc wallet employee, gaining access to session cookies and communication systems. They intercepted a legitimate transaction months later, laundering funds through Tornado Cash mixers, THORChain bridges to Ethereum and Avalanche, and the Huione Guarantee marketplace. The exchange shut down in December

2024 [CoinDesk](https://www.coindesk.com/business/2024/12/02/japanese-crypto-exchange-dmm-bitcoin-to-shut-down-after-305-m-hack) with zero recovery. [Chainalysis +2](https://www.chainalysis.com/blog/crypto-hacking-stolen-funds-2025/) Bitcoin totaled ****\$542.7M in losses**** [CertiK](https://www.certik.com/resources/blog/hack3d-the-web3-security-report-2024) [CertiK](https://www.certik.com/resources/blog/3gKPgxHukAE4bVW5J5Lvnq-hack3d-the-web3-security-report-2024) across multiple incidents according to CertiK.

****Solana drainer infrastructure established significant presence****. The platform hosted a ****6,000+ member Telegram community**** focused on drainer development, with multiple operations including CLINKSINK (hundreds of thousands stolen), Aqua Drainer, and Vanish Drainer. [CCN +4](https://www.ccn.com/news/more-than-6000-people-join-solana-drainer-community/) ScamSniffer tracked ****\$4.17 million stolen from 3,947 victims**** across 2023-2024, though this likely underestimates total losses given unreported incidents. [Google Cloud](https://cloud.google.com/blog/topics/threat-intelligence/solana-cryptocurrency-stolen-clinksink-drainer-campaigns) Address poisoning attacks contributed ****\$3M+** through creating similar-ending addresses** that confuse users copying from transaction history. Solana drainer kits actively sold with pricing from \$250/month to premium models charging \$5,000-10,000 plus 20% commission. [CoinTelegraph](https://cointelegraph.com/news/solana-wallet-drainers-target-users-amid-rise-in-sol-price) [CCN](https://www.ccn.com/news/more-than-6000-people-join-solana-drainer-community/)

****WazirX represents second-largest 2024 incident****. The July attack on India's WazirX exchange resulted in ****\$234.9 million stolen**** (50% of exchange assets) through multi-signature wallet manipulation attributed to Lazarus Group. [The Block +2](https://www.theblock.co/post/331626/crypto-hacks-exploits-2024) The 4-of-6 multi-sig was compromised via smart contract modification attack, with only \$3M recovered as of January 2025. The exchange ceased operations and entered restructuring in Singapore courts. [Crystal Intelligence +2](https://crystalintelligence.com/investigations/expert-analysis-wazirx-hack/)

****State-sponsored attacks dominated large-scale thefts****. North Korea-linked groups stole ****\$1.34 billion across 47 incidents**** in 2024, representing 102.88% increase from 2023's \$660M. [medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) Lazarus Group, TraderTraitor/Jade Sleet, and APT-28 targeted centralized exchanges and high-value accounts using advanced social engineering. The FBI, Department of Defense, and Japanese police confirmed attribution for multiple major incidents. [Chainalysis](https://www.chainalysis.com/blog/crypto-hacking-stolen-funds-2025/) ****78.33% of stolen funds flowed through Tornado Cash**** for laundering, with 82.86% of cases returning funds to mixing services after theft.

****Phishing and private key compromise led attack methods****. CertiK attributed ****\$1.05 billion (44.4%)** to phishing and wallet drainers across 296 incidents averaging \$2.83M but with median loss of just \$207,556, [GlobeNewswire +3](https://www.globenewswire.com/news-release/2025/01/02/3003553/0/en/CertiK-s-Hack3d-Report-2-3-Billion-Lost-to-Scams-Hacks-and-Exploits-in-2024.html) indicating many small-scale attacks alongside major breaches. [Hacken](https://hacken.io/insights/2024-security-report/) Private key compromises accounted for ****\$855.4M (36.2%)**** across 65 incidents, [CertiK +2](https://www.certik.com/resources/blog/hack3d-the-web3-security-report-2024) primarily affecting centralized exchanges. [Invezz](https://invezz.com/news/2025/01/21/tron-wallet-vulnerability-allows-attackers-to-take-control-experts-warn/) Smart contract vulnerabilities caused ****\$214M** across 99 incidents** [medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) per SlowMist. Access control exploits represented ****75% of all hacks**** according to Hacken, with \$1.7B stolen through access control issues. [Medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) [Crowdfund Insider](https://www.crowdfundinsider.com/2025/01/234920-defi-remains-most-frequently-targeted-crypto-and-web3-sector-for-attacks-slowmist-blockchain-security-report/) Rug pulls totaled ****\$106M** across 58 incidents**, with zkSync experiencing \$36.95M and BSC recording the most incidents at 28. [medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) [Crowdfund Insider](https://www.crowdfundinsider.com/2025/01/234920-defi-remains-most-frequently-targeted-crypto-and-web3-sector-for-attacks-slowmist-blockchain-security-report/)

****Drainer-as-a-service market evolved through consolidation****. Pink Drainer held 28% market share in Q1-Q2 before exiting in May 2024. Inferno Drainer dominated with 40-45% market share in Q2-Q3, stealing ****\$110M** in August-September alone**. [Moonlock](https://moonlock.com/wallet-drainer-crypto-theft-2024) Q4 saw Inferno and Angel Drainer combining for 45% share, with new entrant Acedrainer capturing 20% and other operations taking 25%. [medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) The ecosystem deployed ****290,000+ malicious domains**** and maintained 25,000+ active phishing email addresses. [Moonlock](https://moonlock.com/wallet-drainer-crypto-theft-2024) Permit signatures (EIP-2612 token spending approvals) accounted for 56.7% of thefts, while setOwner attacks (smart contract ownership changes) represented 31.9%. [Bleeping Computer +3](https://www.bleepingcomputer.com/news/security/cryptocurrency-wallet-drainers-stole-494-million-in-2024/)

****Recovery rates remained dismally low****. Only ****8.25%** of stolen funds (\$166M)** were recovered in 2024 [medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) despite improved blockchain analytics. Q1 showed the highest recovery at 22% (\$73.9M), but subsequent quarters declined significantly. Tether froze \$540M in USDT associated with

illicit activity, and Circle froze \$13.4M in USDC, [medium](https://slowmist.medium.com/slowmist-2024-blockchain-security-and-anti-money-laundering-annual-report-d7fc94ccfb24) yet these actions preceded most major thefts. The gap between sophisticated tracking capabilities and actual fund recovery highlights limitations in cross-border law enforcement and mixer effectiveness.

Permission models create attack surface differences

The fundamental distinction between blockchain architectures determines which attack vectors exist and what security measures users require.

****EVM chains use contract-based token approvals**.** Ethereum and compatible chains store token balances in smart contract mappings. The ERC-20 standard implements `approve(spender, amount)` allowing contracts to spend tokens on behalf of owners, and `transferFrom(from, to, amount)` enabling approved contracts to move tokens. [Allbridge +2](https://home.core.allbridge.io/compare-standard/difference-swap-trc20-and-erc20) Users commonly approve unlimited amounts (`type(uint256).max`) for convenience. This creates persistent spending permissions that remain active indefinitely until revoked.

[Medium](https://medium.com/puffer-fi/navigating-the-waters-of-erc20-token-approval-and-permit-functions-safeguarding-against-phishing-088b3b07fbf3) Attackers exploit this through phishing sites requesting approvals to malicious contracts, compromised legitimate contracts, and fake dApp frontends that submit approval transactions instead of displayed operations.

[cexplorer](https://cexplorer.io/article/why-is-simple-to-drain-the-ethereum-wallet-unlike-the-cardano-wallet) ****Revocation requires explicit blockchain transactions**** using tools like Revoke.cash to identify and cancel approvals. [Revoke.cash](https://revoke.cash/) [Revoke.cash](https://revoke.cash/learn/approvals/how-to-revoke-token-approvals)

****Solana's delegation system differs critically from EVM**.** SPL tokens support only ****one delegate per token account**** rather than multiple spenders. The delegate address and delegated amount are stored directly in the 165-byte token account structure. New delegations overwrite previous ones, eliminating approval audit trails. Delegates persist even when account balances drop below delegated amounts, and no automatic revocation occurs after delegate use. [Stack

Exchange](https://solana.stackexchange.com/questions/755/are-spl-token-delegates-exclusive-transfer-authorities) [solana](https://spl.solana.com/token) More dangerous than delegation is ****account ownership transfer****—attackers can change the authority controlling a token account entirely, gaining permanent control. The account-based architecture with writable account permissions means signing a transaction grants ALL included programs permission to modify specified accounts.

[blockaid](https://www.blockaid.io/blog/dissecting-toctou-attacks-how-wallet-drainers-exploit-solanas-

transaction-timing) [Solana Wiki](https://solana.wiki/docs/solidity-guide/transactions/) Tools like Revoka and Solrevoker help manage delegations, but account ownership transfers require more sophisticated monitoring.

****Cosmos eliminates approvals through architecture**.** SDK-based chains have no token approval mechanism. IBC token transfers (ICS20 standard) lock tokens on source chains and mint IBC-denominated versions on destinations without any pre-authorization system. Each cross-chain transfer requires explicit user signature. CosmWasm smart contracts cannot gain persistent spending rights over user wallets. The authz module enables delegation of specific message types (staking, governance, sending) with optional spend limits and expiration times, providing more granular control than EVM approvals. [Cosmos Network](https://tutorials.cosmos.network/academy/2-cosmos-concepts/14-interchain-security.html) Users revoke authorizations via command-line or chain-specific wallet interfaces: `'simd tx authz revoke [grantee] [msg-type-url] --from=[granter]'`. This design eliminates approval-based wallet draining entirely, shifting security concerns to IBC vulnerabilities and protocol-level exploits. [GitHub +2](https://github.com/cosmos/cosmos-sdk/blob/main/docs/architecture/adr-030-authz-module.md)

****Cardano validator scripts replace approval contracts**.** Extended UTXO architecture stores value in discrete outputs locked at script addresses rather than account balances. Plutus smart contracts are validators that evaluate whether transactions meet spending conditions—they cannot initiate transfers. Users construct transactions consuming UTXOs; contracts approve or reject based on datum (data attached to UTXO), redeemer (data provided by spender), and context (transaction and blockchain information). [SDLC Corp](https://sdlccorp.com/post/common-issues-with-trc-20-tokens/) ****No approval system exists**** because contracts don't need permission to spend user funds—users explicitly create spending transactions that contracts merely validate. [Cardano Developer Portal](https://developers.cardano.org/docs/smart-contracts/) [Emurgo](https://www.emurgo.io/press-news/why-crypto-projects-built-on-cardano-are-more-secure/) Native tokens defined by minting policy scripts inherit ledger security without separate contract logic. [IOHK](https://iohk.io/blog/posts/2025/05/09/native-tokens-in-the-extended-utxo-model/) [cexplorer](https://cexplorer.io/article/why-is-simple-to-drain-the-ethereum-wallet-unlike-the-cardano-wallet) The collateral requirement provides economic guarantee against malicious transactions. This architecture makes it architecturally impossible for contracts to drain funds without explicit user transaction creation for each spend. [Cardano Developer Portal](https://developers.cardano.org/docs/smart-contracts/)

****NEAR's function-call keys revolutionize permission model**.** Accounts support multiple access keys with two types: full-access keys granting complete control, and function-call keys restricted to specific contracts with gas allowances. When connecting to dApps, NEAR creates function-call keys that only interact with that dApp's contract, have limited gas budgets, cannot transfer NEAR tokens, and deplete allowances with use. [near](https://docs.near.org/concepts/protocol/access-keys)

[Near](https://docs.near.org/concepts/protocol/access-keys) ****This is superior to EVM approvals**** because each dApp has isolated permissions, compromised keys don't affect other services, gas limits prevent indefinite draining, and token transfers always require explicit user confirmation. Users revoke access by removing the key from their account via wallet settings.

[Near](https://docs.near.org/concepts/protocol/access-keys) The protocol-level implementation means security doesn't depend on smart contract code correctness. Account recovery through email/SMS (now deprecated after 2022 vulnerability) represented the primary security concern rather than permission exploitation.

****Move chains use object capabilities instead of approvals**.** Aptos stores resources directly in user accounts rather than contract mappings. Function signatures requiring '&signer' parameter ensure only the resource owner can authorize moves. No allowance system exists—each transaction requires the owner's signature. Sui extends this with four ownership types: owned by address (exclusive control enabling parallel execution), owned by object (parent-child relationships), immutable (no owner, cannot modify), and shared (accessible by anyone requiring consensus). The type system prevents unauthorized access through abilities (key, store, copy, drop) controlling value behavior. ****Resources cannot be copied or dropped arbitrarily**** unless explicitly designed with those abilities, preventing accidental duplication or loss at language level. [Pontem](https://docs.pontem.network/02.-move-language/lang) Security issues arise from excessive abilities (e.g., flash loan receipts with drop ability allowing borrowers to skip repayment) or improper object sharing rather than approval mechanisms.

****Algorand requires opt-in but no persistent approvals**.** ASA tokens are layer-1 protocol assets, not smart contracts. Accounts must explicitly opt-in via zero-amount self-transfer before receiving any asset, preventing spam and unwanted token attacks. [Algorand](https://dev.algorand.co/getting-started/ethereum-to-algorand/) [Algorand Technologies](https://algorandtechnologies.com/news/how-to-tokenize-assets-on-blockchain/) Smart contracts written in TEAL execute as stateful applications (with storage) or stateless smart signatures (logic signs transactions directly).

[Algorand](https://developer.algorand.org/docs/features/ascl/)

[Algorand](https://developer.algorand.org/docs/get-details/dapps/smart-contracts/apps/) ****No approval mechanism exists**** for persistent spending permissions. Each transaction requires explicit signature or smart signature logic evaluation. The rekeying feature allowing accounts to rotate spending keys while maintaining addresses creates unique vulnerability—contracts failing to validate the rekey_to field can be permanently hijacked. [CoinInspect Security +2](https://www.coinspect.com/blog/algorand-rekeying-attacks/) Tools like Pera Wallet manage ASA opt-ins but no revocation system is needed since no approvals are granted. [algorand](https://developer.algorand.org/docs/get-details/asa/)

****Tron replicates EVM vulnerabilities**.** The Tron Virtual Machine's Solidity compatibility means TRC-20 tokens implement identical approve() and transferFrom() functions as ERC-20. The same attack vectors

exist: unlimited approvals, malicious contracts, phishing sites requesting excessive permissions. [B2BINPAY+3](<https://b2binpay.com/en/news/what-is-trc20-understanding-trons-token-standard-and-its-role-in-defi>) However, Tron adds the **UpdateAccountPermission** function for multi-signature management. This enables legitimate multi-sig setups but also allows attackers with compromised keys to modify permission structures, adding their own keys and locking out legitimate owners.

[OneSafe](<https://www.onesafe.io/blog/tron-wallet-exploit-updateaccountpermission>) [The Currency analytics](<https://thecurrencyanalytics.com/altcoins/over-14500-tron-wallets-at-risk-of-silent-hijacking-exposing-31-5-million-156658>) The 100 TRX fee provides minimal barrier.

[Medium](<https://medium.com/coinmonks/top-5-crypto-hacks-of-2024-more-than-2-billion-lost-36crypto-559a481eff9c>) [CoinTelegraph](<https://cointelegraph.com/news/tron-addresses-risk-silent-hijacking>) Tools like Revoke.cash support Tron for managing TRC-20 approvals, while block explorers like TronScan include approval checkers. The EVM compatibility means Tron requires the highest user vigilance among non-EVM chains.

****TON's jetton validation prevents some exploits**.** The two-contract jetton system (master contract plus individual jetton wallet contracts per user) deterministically calculates wallet addresses from owner addresses and jetton master addresses. Contracts receiving jetton transfers MUST validate that the sender's jetton wallet address matches the expected calculated address to prevent fake token attacks. [The Open Network](<https://docs.ton.org/v3/guidelines/smart-contracts/security/common-vulnerabilities/>) [ton](<https://docs.ton.org/v3/guidelines/smart-contracts/security/common-vulnerabilities/>) The message-based actor architecture requires impure modifier on security functions or the compiler optimizes them away. [ton](<https://docs.ton.org/v3/guidelines/smart-contracts/security/common-vulnerabilities/>) No persistent approval system exists—each transaction requires explicit signing—but the Telegram integration creates social engineering attack surfaces through bots requesting wallet connections. Limited security tooling compared to mature chains increases phishing vulnerability.

Protection strategies vary dramatically by architecture

Security tools and user vigilance requirements differ fundamentally across blockchain architectures, with some chains requiring constant permission monitoring while others make such tools architecturally unnecessary.

****Solana demands highest vigilance with specialized revocation tools**.** Users should check active delegations monthly using Revoka (revoka.debyth.com), Solrevoker (solrevoker.com), or Famous Fox Federation Revoker. These tools connect wallets to display all SPL token delegations and enable one-click or selective revocation. Transaction simulators in Phantom and Solflare wallets show expected token

changes before signing, though T0CT0U attacks can bypass simulation guarantees.

[Blockaid](https://www.blockaid.io/blog/dissecting-toctou-attacks-how-wallet-drainers-exploit-solanas-transaction-timing) [blockaid](https://www.blockaid.io/blog/dissecting-toctou-attacks-how-wallet-drainers-exploit-solanas-transaction-timing) ****Account ownership transfer monitoring is critical****—these are more dangerous than simple delegations because they permanently change account control. Hardware wallet integration (Ledger with Solflare/Phantom) provides additional protection for significant holdings. The 400ms block finality [Solana](https://solana.com/docs/core/transactions) means attacks execute extremely quickly once initiated. Regular audits should verify no unexpected programs have authority over token accounts and no account ownership has been transferred to unknown addresses.

****Tron requires EVM-style approval management****. Revoke.cash supports Tron network for checking and revoking TRC-20 token approvals. TronScan and 0KLink explorers include token approval checker features. Users should regularly audit approvals, limit approvals to specific amounts rather than unlimited, and revoke immediately after completing DeFi activities. [ECOS](https://ecos.am/en/blog/trc20-token-standard-understanding-trc20-in-the-tron-network/) The ****UpdateAccountPermission vulnerability requires monitoring permission structures**** via block explorers to detect unauthorized key additions. [Medium](https://medium.com/coinmonks/top-5-crypto-hacks-of-2024-more-than-2-billion-lost-3bcrypto-559a481eff9c) [Yellow](https://yellow.com/news/tron-wallet-security-flaw-leaves-over-14000-wallets-vulnerable-research) TronLink wallet with hardware wallet integration (Ledger, Trust Wallet) provides baseline security. [CoinGecko](https://www.coingecko.com/learn/top-ton-wallets-jettons-crypto) The fast 3-second block time means compromises execute rapidly. [B2BINPAY](https://b2binpay.com/en/news/main-differences-between-erc-20-and-trc-20-which-one-to-choose-for-your-purposes) [Chain](https://chain.compare/ton-vs-tron) Security firms GoPlus, SharkTeam, and BlockApex provide auditing services for smart contracts. The EVM compatibility means Tron users face identical risks to Ethereum users requiring similar diligence. [Medium](https://nexera.medium.com/240807-incident-guide-to-revoking-token-approvals-be8559e204b9) [Revoke.cash](https://revoke.cash/learn/approvals/how-to-revoke-token-approvals)

****Cosmos chains use command-line revocation****. The authz module manages authorizations via CLI commands: `'simd tx authz revoke [grantee] [msg-type-url] --from=[granter]'`. Users query active grants with `'simd query authz grants [granter] [grantee] [msg-type-url]'`. [Securities.io](https://www.securities.io/investing-in-cosmos-atom-everything-you-need-to-know/) Keplr wallet displays some authorizations in certain implementations but no universal graphical revocation tool exists. ****Authorization grants should have expiration times set**** when created and spend-limit parameters when possible. The lack of persistent approvals means lower revocation urgency, but staking delegations and send authorizations warrant regular review. Each Cosmos chain may implement authz differently, requiring chain-specific knowledge. The defense-in-depth approach with rate limits and permissioned contract deployment on many chains provides additional protection beyond user vigilance. [GitHub](https://github.com/cosmos/cosmos-sdk/blob/main/x/authz/README.md)

[Cosmos](https://docs.cosmos.network/v0.46/modules/autlz/05_client.html)

****Aptos and Sui use connected app management****. Martian Wallet (supporting both chains) includes transaction preview showing wallet balance changes before confirmation, Trusted Apps management for viewing and revoking dApp connections, and built-in security scanners. Revocation means disconnecting apps from wallet settings rather than canceling spending permissions. The ****Move language architecture prevents unauthorized transfers**** at type system level—resources require owner signature for movement regardless of app connections. Users should preview every transaction's balance changes, regularly audit connected apps, revoke access to unused services, and verify dApp legitimacy before connection. Hardware wallet support remains limited compared to EVM chains. The lack of persistent approvals means connected app monitoring suffices rather than continuous permission auditing.

[Martianwallet](https://martianwallet.xyz/faq/) [Martianwallet](https://martianwallet.xyz/)

****Cardano requires minimal approval-related vigilance****. No revocation tools exist because ****no approval mechanism exists**** in the eUTXO model. Security comes from transaction-by-transaction review rather than permission management. Wallets (Daedalus, Yoroi, Nami, Eternl, Flint) require explicit signing for each spend with complete UTXO details visible. Users should review which UTXOs transactions consume, understand collateral requirements for smart contract interactions, verify smart contract addresses, and check datum parameters. Native tokens eliminate ERC-20-style vulnerabilities entirely. [IOHK +2](https://iohk.io/blog/posts/2025/05/09/native-tokens-in-the-extended-utxo-model/) The validator script architecture means contracts cannot autonomously drain funds—users must explicitly construct spending transactions for each operation. [Cardano Developer Portal](https://developers.cardano.org/docs/smart-contracts/) IOHK's smart contract certification program helps identify audited dApps. The architecture makes Cardano the least vigilant-requiring chain for approval-related security.

****NEAR's access key system needs periodic review****. Function-call keys for dApp connections appear in wallet settings under Connected Apps or Authorized Apps. Users should deauthorize unused apps, monitor gas allowances on active keys, investigate any apps requesting full-access keys (rare and suspicious), and verify transaction details even with limited keys since allowances enable multiple operations. [near +2](https://docs.near.org/concepts/protocol/access-keys) Hardware wallet integration with Ledger provides additional security. [CoinGecko](https://www.coingecko.com/learn/top-ton-wallets-jettons-crypto) The ****function-call key design is inherently more secure**** than EVM approvals because keys are app-specific with gas limits, token transfers require explicit confirmation, and compromised keys don't affect other dApps. Account recovery via email/SMS (deprecated after 2022 vulnerability) represented the main attack vector rather than permission exploitation. [Decrypt](https://decrypt.co/106819/near-protocol-wallet-breach-exposed-private-keys) NEAR Wallet, Meteor Wallet, and HERE Wallet include security alerts for suspicious activity.

****Polkadot and Algorand use transaction-by-transaction signing**.** Neither chain has persistent approval mechanisms requiring dedicated revocation tools. Polkadot's parachains implement independent security models; Polkadot.js, Talisman, and SubWallet require explicit signing for each extrinsic. Users should review XCM cross-chain messages carefully, verify parachain legitimacy, and understand proxy account permissions if used. Algorand's opt-in model for ASA tokens prevents unwanted assets.

[Algorand](https://dev.algorand.co/getting-started/ethereum-to-algorand/) [Algorand Technologies](https://algorandtechnologies.com/news/how-to-tokenize-assets-on-blockchain/) Pera Wallet, MyAlgo Wallet (post-2023-incident), and Defly Wallet manage assets with explicit transaction confirmation. The ****rekeying feature requires validation****—any transaction including `rekey_to` fields warrant extreme scrutiny as account control can be permanently transferred. Smart signatures must explicitly check `rekey_to` equals `ZeroAddress` to prevent hijacking. [CoinInspect Security](https://www.coininspect.com/blog/algorand-rekeying-attacks/) Both chains prioritize hardware wallets for significant holdings. [Polkadot](https://wiki.polkadot.com/learn/learn-bridges/)

****TON lacks mature security tooling**.** No dedicated revocation tools exist; security relies on wallet app permission management. Tonkeeper, Tonhub, MyTonWallet, and HOT Wallet require transaction confirmation for all operations. [CoinGecko](https://www.coingecko.com/learn/top-ton-wallets-jettons-crypto) Users should verify bot/dApp legitimacy before connecting via Telegram, review every transaction, disconnect from unused dApps, and exercise extreme caution with Telegram-based interactions.

[Wallet](https://help.wallet.tg/article/3bb-secure-your-ton-space) The ****influx of crypto novices**** from Telegram games creates high phishing vulnerability. Scam Sniffer detected surge in TON phishing sites with drainer scripts sold for \$300-1,000 targeting Tonkeeper specifically.

[AirdropAlert](https://airdropalert.com/blogs/ton-wallet-drainers/) [Airdrop Alert](https://airdropalert.com/blogs/ton-wallet-drainers/) The asynchronous actor model and custom architecture mean less security tooling than established chains. Social engineering through Telegram bots represents primary attack vector rather than technical exploits.

****User vigilance requirements ranked by architecture**.** Highest vigilance needed: Tron (EVM-compatible with additional permission exploit), Solana (account ownership transfers plus delegations with fast execution). Medium vigilance: Cosmos chains (explicit authorizations requiring CLI management), Polkadot (parachain-specific risks), Aptos/Sui (connected app monitoring), TON (Telegram integration social engineering). Lowest vigilance: NEAR (function-call keys with gas limits), Cardano (no approval mechanism), Algorand (opt-in model with no persistent approvals). The correlation is clear: ****chains with persistent permissions require constant monitoring****, while chains architected to eliminate approvals need only transaction-by-transaction review.

Architectural security trade-offs shape ecosystem resilience

Different blockchain designs reveal fundamental tension between flexibility, user experience, and security—each making conscious trade-offs that determine vulnerability patterns.

****Account-based models enable attacks but simplify state**.** Ethereum, Tron, and NEAR use account balances rather than UTXOs. This creates intuitive mental models (users understand “balance in account”) and enables efficient state lookups. However, account models naturally support persistent permissions—contracts can be authorized to modify balances at future times. EVM chains weaponize this flexibility through unlimited approvals enabling wallet drainers. Solana’s account model allows even more dangerous account ownership transfers. NEAR mitigates through function-call keys with gas limits, demonstrating that account models can implement secure permission systems through architectural choices rather than requiring UTXO models.

****UTXO models eliminate approvals but complicate concurrency**.** Cardano and Algorand (for some operations) use unspent transaction outputs. [cexplorer](https://cexplorer.io/article/why-is-simple-to-drain-the-ethereum-wallet-unlike-the-cardano-wallet) Each spend consumes specific UTXOs entirely, creating new outputs as change. Smart contracts validate whether spending conditions are met but cannot initiate transfers. This ****architecturally prevents approval-based attacks**** since no persistent permissions exist—each spend requires explicit user transaction creation. However, UTXO contention creates scaling challenges when multiple users attempt to consume the same UTXO simultaneously (common in DEX liquidity pools). Cardano addresses this through careful protocol design distributing state across multiple UTXOs. The deterministic execution enables perfect off-chain simulation since outcomes depend only on inputs, not global state.

****Resource-oriented programming prevents duplication**.** Aptos and Sui’s Move language treats digital assets as first-class resources with linear types. Resources cannot be copied (preventing double-spending at language level), cannot be dropped arbitrarily (preventing loss), and must be explicitly moved between accounts. The type system encodes ownership, eliminating complex access control logic. This prevents reentrancy attacks (resources are moved, not referenced), integer overflow (strict arithmetic), and unauthorized access (ownership in types). However, the ****Cetus \$220-260M exploit**** demonstrated that protocol-level vulnerabilities (arithmetic overflow in liquidity math, oracle manipulation) still threaten even theoretically secure languages. The advantage is fewer vulnerability classes, not immunity to all exploits.

****Cross-chain communication creates universal vulnerability**.** Cosmos IBC, Polkadot XCM, and bridges

between ecosystems represent attack surfaces regardless of underlying chain security. Over \$2 billion has been lost to bridge exploits historically, with 60%+ of major hacks targeting centralized bridges. The ****DMM Bitcoin \$305M theft**** and ****WazirX \$234.9M exploit**** both involved centralized custody despite being “Bitcoin” and “Ethereum” incidents respectively. Cosmos’s native IBC with light-client verification provides superior security to trusted bridges, yet still requires careful trust configuration. The architectural lesson: ****cross-chain security is only as strong as the weakest link****—a perfectly secure blockchain becomes vulnerable when bridging to less secure systems.

****Permission granularity inversely correlates with UX friction****. EVM unlimited approvals maximize convenience—users approve once, interact repeatedly without re-signing. This drove DeFi adoption but enabled \$494M in drainer losses. NEAR’s function-call keys with gas limits provide security through granularity but require users to re-approve when allowances deplete. Cardano’s complete absence of approvals maximizes security at cost of requiring transaction signatures for each interaction. The most successful chains balance security with usability: Aptos/Sui maintain security through Move’s type system while enabling multi-operation transactions, Cosmos chains support authorization modules with optional expirations for power users. Universal truth: ****security and convenience exist in tension****; architecture determines where that balance is struck.

****Newer chains leverage hindsight but lack battle-testing****. Aptos (2022), Sui (2023), and TON (2018 original, 2021 relaunch) designed security models learning from Ethereum’s failures. Move language specifically prevents reentrancy, Algorand’s pure proof-of-stake aims to solve security trilemma, NEAR’s access keys improve upon unlimited approvals. Yet these chains have ~1,000–10,000x less transaction history than Ethereum. The ****MyAlgo \$9.2M hack**** and ****Cetus \$220–260M exploit**** revealed vulnerabilities in theoretically superior designs. Ethereum’s security comes partly from having survived countless attack attempts that exposed and resolved vulnerabilities. The maturity trade-off: established chains have better security tooling and discovered vulnerabilities, newer chains have architectures designed to prevent known attack classes but face unknown vulnerabilities.

****Drainer migration follows tooling and value density****. Wallet draining operations are economically rational businesses that target chains offering optimal risk-reward. Ethereum dominated with 85.3% of drainer losses because it has the highest value density (\$465–748M in total losses), most users unfamiliar with security best practices, and established drainer-as-a-service infrastructure. As Ethereum security improved (Blockaid integration with MetaMask, Wallet Guard adoption), drainers migrated to Solana (emerging drainer ecosystem) and TON (crypto novices from Telegram games). Tron’s \$31.5M vulnerability attracted targeted exploitation. Cosmos, Cardano, Polkadot, and Algorand remain largely untargeted due to ****lower transaction volumes, architectural security advantages, and smaller user bases****. The pattern suggests that no chain is inherently immune—attackers follow liquidity and vulnerable users.

****State-sponsored attacks target centralized custody regardless of chain****. North Korea's \$1.34 billion in 2024 thefts primarily exploited private key compromises at centralized services. The ****DMM Bitcoin \$305M**** and ****WazirX \$234.9M**** incidents bypassed Bitcoin and Ethereum's security through social engineering targeting exchange employees. TraderTraitor used fake job offers with malicious Python scripts sent via LinkedIn, gaining session cookies and communication access months before intercepting legitimate transactions. This reveals that ****custody models matter more than blockchain architecture**** for large-scale thefts. Decentralized wallet security varies by chain design (Cardano safer than EVM), but centralized exchange security depends on operational security, multi-signature configurations, and employee training across all chains equally.

The evidence suggests that while certain architectures (Cardano's UTXO, NEAR's access keys, Move's resource model) provide superior security against wallet draining compared to EVM chains, no design eliminates all vulnerabilities. Protocol-level bugs (Cetus), centralized custody failures (DMM Bitcoin), social engineering (WazirX), and emerging attack vectors (Tron's silent hijacking) affect even well-designed chains. The most resilient ecosystems combine architectural security advantages with mature tooling, active security research communities, and user education programs—qualities that develop through years of operation rather than purely from initial design choices.